Informe de análisis dinámico (JaCoCo)

Proyecto inicial Slot Machine, Cierre
Juan David Guarguati Guarguati, Juan Pablo Suárez Rubiano
Octubre 2026

1. Qué se midió

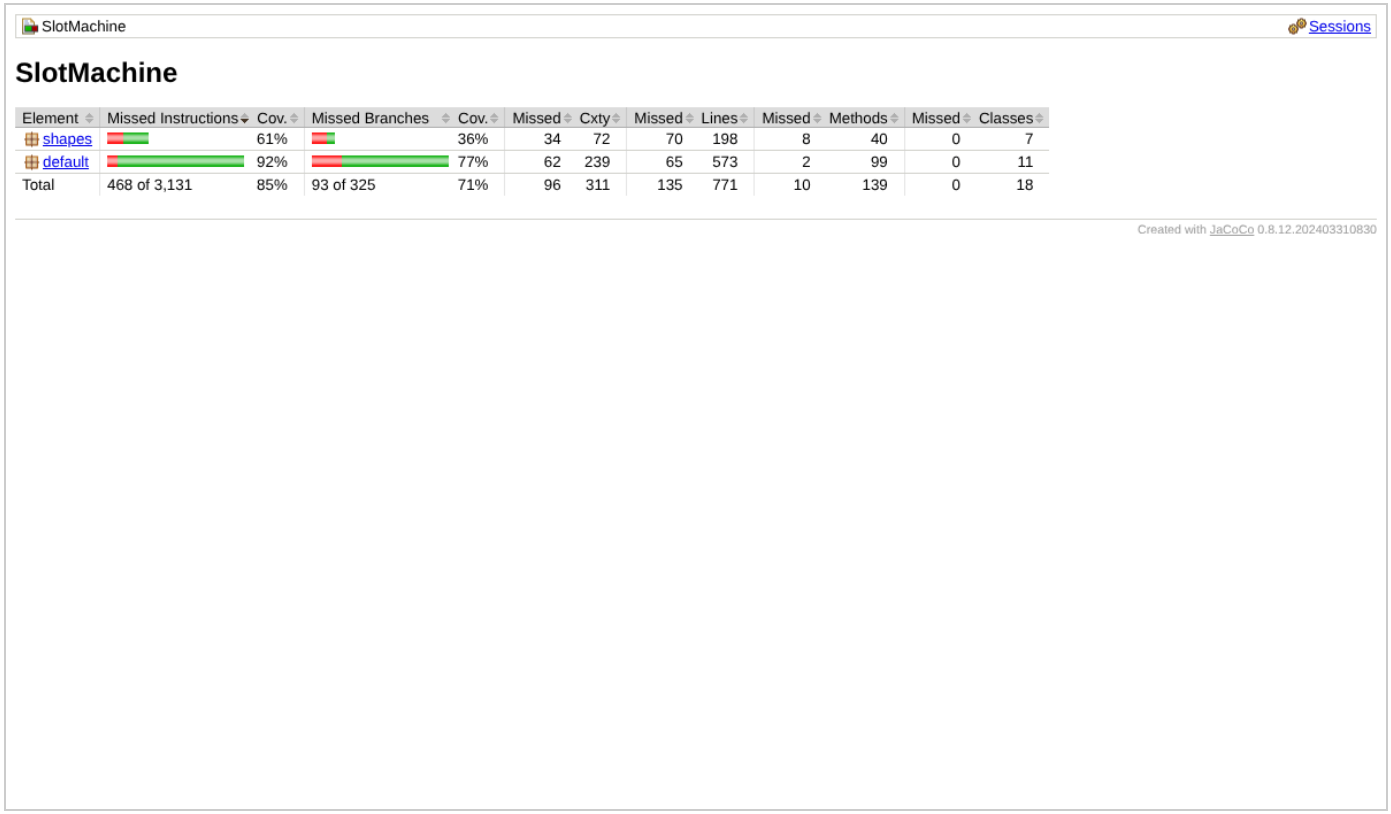
Se midió el **cubrimiento de código** de las pruebas de unidad con **JaCoCo 0.8.12**, integrado en Maven (`pom.xml`).

- **Pruebas que cuentan:** `SlotMachineC1Test`, `C2Test`, `C3Test`, `C4Test`, `CC2Test`, `CC4Test`, `ContestTest` y `ContestCTest`.
- **Prueba que no cuenta:** `SlotMachineAcceptanceTest`. Necesita a una persona que mire la pantalla y responda, así que se ejecuta aparte desde VS Code.
- **Cómo se genera:** en VS Code, terminal, `mvn clean test`. El informe queda en `target/site/jacoco/index.html`.
- **Meta del enunciado:** más del **75 %** de cubrimiento de código.

2. Resultado inicial

Proyecto recién pasado a VS Code, sin ningún cambio. Se ejecutaron 73 pruebas y todas pasan.

Medida	Cubrimiento
Instrucciones	**85,1 %**
Ramas (if / switch)	71,4 %
Líneas	82,5 %
Métodos	92,8 %



SlotMachine

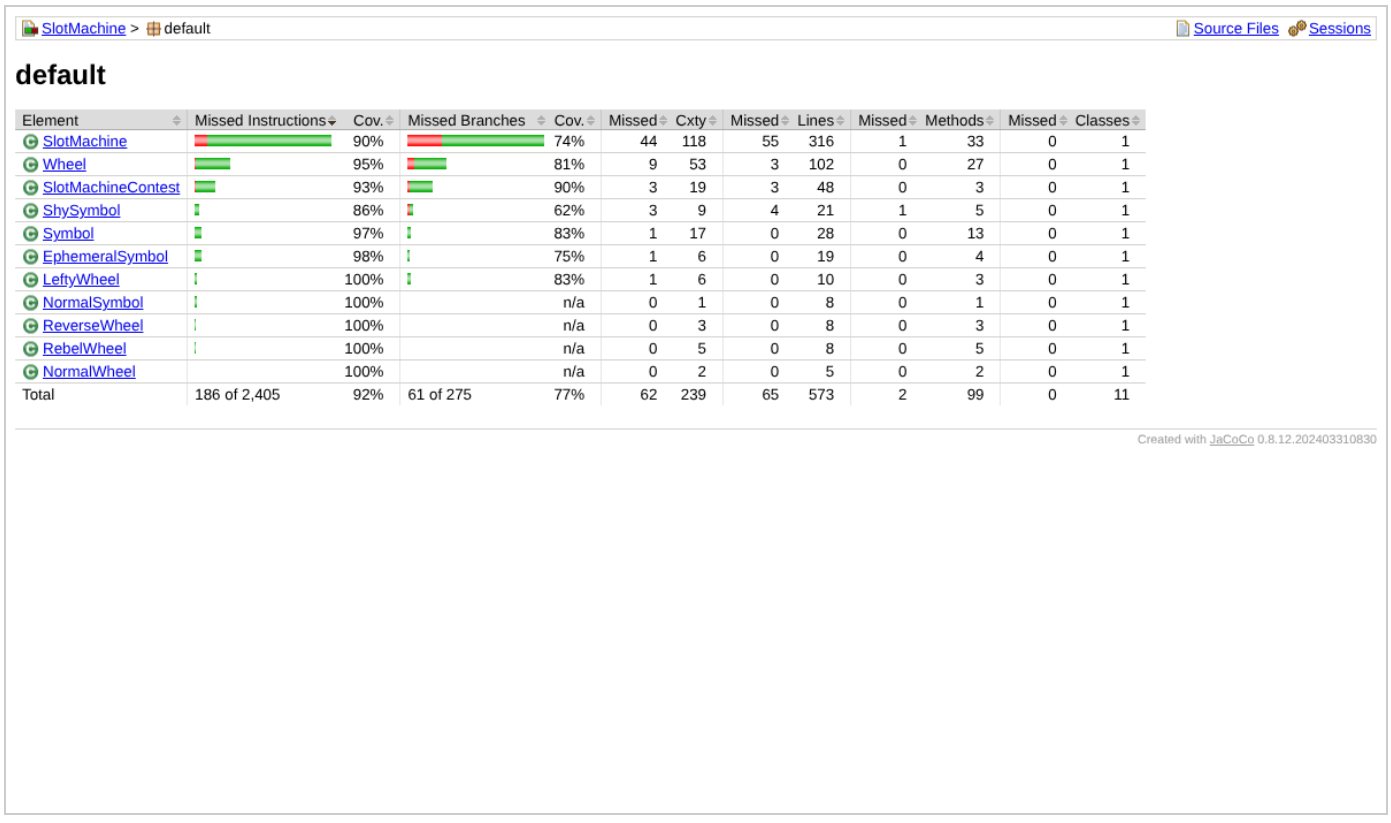
Sessions

SlotMachine

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
shapes		61%		36%	34	72	70	198	8	40	0	7
default		92%		77%	62	239	65	573	2	99	0	11
Total	468 of 3,131	85%	93 of 325	71%	96	311	135	771	10	139	0	18

Created with JaCoCo 0.8.12.202403310830

Resultado inicial (resumen)



SlotMachine > default

Source Files Sessions

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
SlotMachine		90%		74%	44	118	55	316	1	33	0	1
Wheel		95%		81%	9	53	3	102	0	27	0	1
SlotMachineContest		93%		90%	3	19	3	48	0	3	0	1
ShySymbol		86%		62%	3	9	4	21	1	5	0	1
Symbol		97%		83%	1	17	0	28	0	13	0	1
EphemeralSymbol		98%		75%	1	6	0	19	0	4	0	1
LeftyWheel		100%		83%	1	6	0	10	0	3	0	1
NormalSymbol		100%		n/a	0	1	0	8	0	1	0	1
ReverseWheel		100%		n/a	0	3	0	8	0	3	0	1
RebelWheel		100%		n/a	0	5	0	8	0	5	0	1
NormalWheel		100%		n/a	0	2	0	5	0	2	0	1
Total	186 of 2,405	92%	61 of 275	77%	62	239	65	573	2	99	0	11

Created with JaCoCo 0.8.12.202403310830

Resultado inicial por clase (paquete principal)

Lo que se vio:

1. La meta de instrucciones ya se cumplía, pero **las ramas estaban en 71,4 %**, por debajo del 75 %.

2. Faltaban dos clases de prueba que pide el enunciado: **`SlotMachineC1Test`** y **`SlotMachineC3Test`**.
3. En `SlotMachine` (90 %) no se probaban varios casos de error: borrar una rueda sin ruedas, borrar un color que no existe, repetir un color, `isJackpot()` con una rueda vacía, `SlotMachine(n)` con n fuera de 3..50 y `spin(wheel, steps)` con pasos negativos.
4. Lo menos cubierto está en `shapes` (`Shape`, `Circle`, `Triangle`). Son métodos que vienen del proyecto de ejemplo de BlueJ y el simulador no usa, como `moveRight()`, `moveUp()` o `slowMoveHorizontal()`.

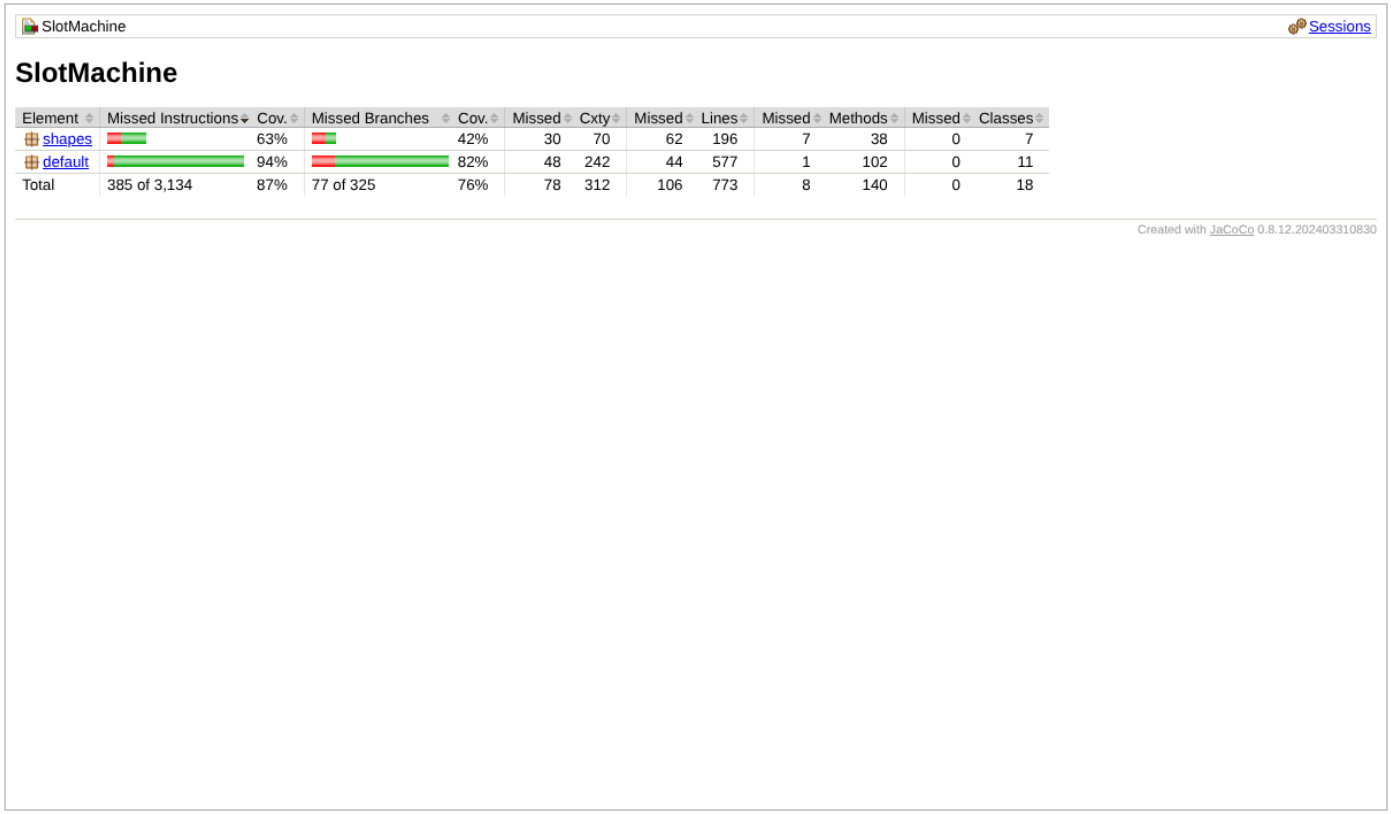
3. Decisiones tomadas

#	Decisión	Por qué
1	Crear **`SlotMachineC1Test`** (17 pruebas)	Lo pide la entrega. Cubre los requisitos 1 a 8: crear, agregar y eliminar ruedas y símbolos, girar, consultar, jackpot, visibilidad y terminar. Incluye los **casos de error** que no se probaban.
2	Crear **`SlotMachineC3Test`** (9 pruebas)	Lo pide la entrega. Cubre `SlotMachine(n)` (límites 3 y 50, n inválido, colores distintos), `spin(wheel, steps)` cíclico y con pasos negativos, y `distinctSymbols()`.
3	**No** escribir pruebas para los métodos de `shapes` que el simulador no usa	Probarlos solo subiría el número sin probar nada del simulador. Se dejan como están porque son parte de la librería `shapes` del curso.
4	**No** contar la prueba de aceptación	No es automática: depende de que una persona responda “Sí”.
5	Ejecutar `simulate` con pantalla	`SlotMachineContestCTest` llama a `simulate(3)`, que abre el lienzo. En VS Code se ve la máquina y el aviso de JACKPOT, que hay que cerrar con **Aceptar**.

4. Resultado final

Se ejecutaron 99 pruebas de unidad y todas pasan.

Medida	Inicial	Final
Instrucciones	85,1 %	**87,7 %**
Ramas	71,4 %	**76,3 %**
Líneas	82,5 %	**86,3 %**
Métodos	92,8 %	**94,3 %**



SlotMachine

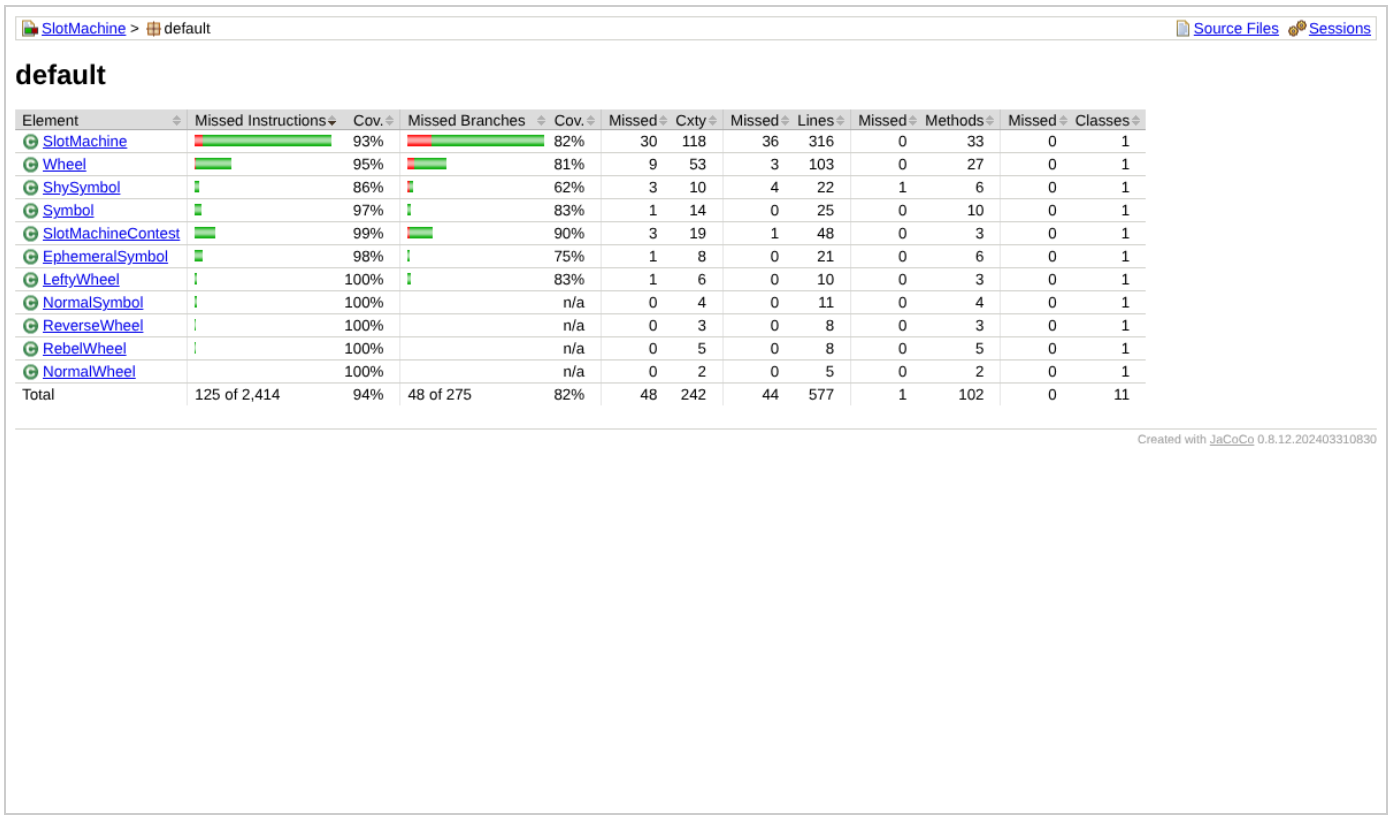
Sessions

SlotMachine

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
shapes		63%		42%	30	70	62	196	7	38	0	7
default		94%		82%	48	242	44	577	1	102	0	11
Total	385 of 3,134	87%	77 of 325	76%	78	312	106	773	8	140	0	18

Created with JaCoCo 0.8.12.202403310830

Resultado final (resumen)



SlotMachine > default

Source Files Sessions

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
SlotMachine		93%		82%	30	118	36	316	0	33	0	1
Wheel		95%		81%	9	53	3	103	0	27	0	1
ShySymbol		86%		62%	3	10	4	22	1	6	0	1
Symbol		97%		83%	1	14	0	25	0	10	0	1
SlotMachineContest		99%		90%	3	19	1	48	0	3	0	1
EphemeralSymbol		98%		75%	1	8	0	21	0	6	0	1
LeftyWheel		100%		83%	1	6	0	10	0	3	0	1
NormalSymbol		100%		n/a	0	4	0	11	0	4	0	1
ReverseWheel		100%		n/a	0	3	0	8	0	3	0	1
RebelWheel		100%		n/a	0	5	0	8	0	5	0	1
NormalWheel		100%		n/a	0	2	0	5	0	2	0	1
Total	125 of 2,414	94%	48 of 275	82%	48	242	44	577	1	102	0	11

Created with JaCoCo 0.8.12.202403310830

Resultado final por clase (paquete principal)

Conclusión: se cumple la meta. Las clases propias del simulador tienen **94 %** de cubrimiento de instrucciones y el proyecto completo **87,7 %**. Todas las medidas quedaron por encima del 75 %, incluidas las ramas.

> Nota: `SlotMachine(n)` y `spin()` son aleatorios. Por eso el porcentaje de `SlotMachineContest` puede variar uno o dos puntos entre una ejecución y otra.